

CENTRAL UNIVERSITY OF ECUADOR

FACULTY OF ENGINEERING AND APPLIED SCIENCES

INFORMATION SYSTEMS ENGINEERING



Technical Report: Architecture and Implementation of UCE Lab Management System

Course:

Distributed Programming

Professor:

Eng. Juan Pablo Guevara Gordillo

Student(s):

Bryan Fabricio Chileno Agualongo

Academic Period 2026-2026

Quito – Ecuador

Contenido

1. Introduction	4
2. Objectives.....	4
3. General System Architecture	5
4. Technologies Used.....	6
5. Monorepo Architecture	6
6. Frontend.....	6
7. API Gateway	7
8. Microservices.....	8
8.1. API Gateway	8
8.2. Auth Service:.....	8
8.3. Reservation Service:	8
8.4. Laboratory Service:	9
8.5. Notification Service:	9
8.6. Catalog Service:	9
8.7. Incident Service:	10
8.8. Logger Service:	10
8.9. Payment Service:	10
8.10. AI Service (AWS Lambda):	10
9. Cloud Infrastructure.....	10
10. Containerization	12
11. Continuous Integration and Deployment (CI/CD)	12
12. Security	12
13. Communication Between Components.....	15
14. Data Persistence	15
15. Observability	15

16. Scalability and High Availability	16
17. Complete System Operation	16
18. Architectural Decisions.....	17
19. Risks and Limitations	17
20. Project Metrics	18
21. AWS Infrastructure Budget Estimate	19
22. Implemented Best Practices	21
23. Possible Future Improvements	21
24. Conclusions	21
Diagram 1. General Architecture	5
Diagram 2. Sequence Diagram - Transactional Outbox	9
Diagram 3. Cloud HA Architecture	11
Diagram 4. Use Case Diagram - Detailed RBAC.....	14

Technical Report: Architecture and Implementation of UCE Lab Management System

1. Introduction

This report exhaustively documents the architecture, design, and implementation of the UCE Lab Management System, a platform oriented towards the academic administration of laboratories, reservation management, incident control, and notifications. The system has been conceived under a modern software engineering approach, adopting an event-driven microservices topology. Through cloud deployment on Amazon Web Services (AWS) using Infrastructure as Code (IaC) and the extensive use of containers, the solution guarantees a high degree of scalability, maintainability, and resilience. The purpose of this document is to detail the internal structure of its components and justify the architectural decisions against the code deployed in production.

2. Objectives

The primary objective is to present a precise technical radiograph of the system based strictly on its actual implementation, outlining the flow of information from the user interface to persistence in databases.

Specific objectives include the analysis of synchronous and asynchronous communication, the deployment model in AWS, the distributed security strategy, the consolidation of operational resilience patterns, and the quantification of project metrics.

3. General System Architecture

The system implements a distributed architecture based on microservices, decoupling the different business domains. The topology is governed by the **Backend for Frontend (BFF)** or API Gateway pattern, which centralizes HTTP traffic coming from the **Single Page Application (SPA)**.

Architecture promotes autonomy through the **Database-per-service** pattern, where each domain governs its own relational or document schema. To guarantee eventual consistency and operational fluidity, choreographic asynchronous communication is used through a message bus (RabbitMQ), mitigating the temporal coupling typical of architectures based solely on REST requests.

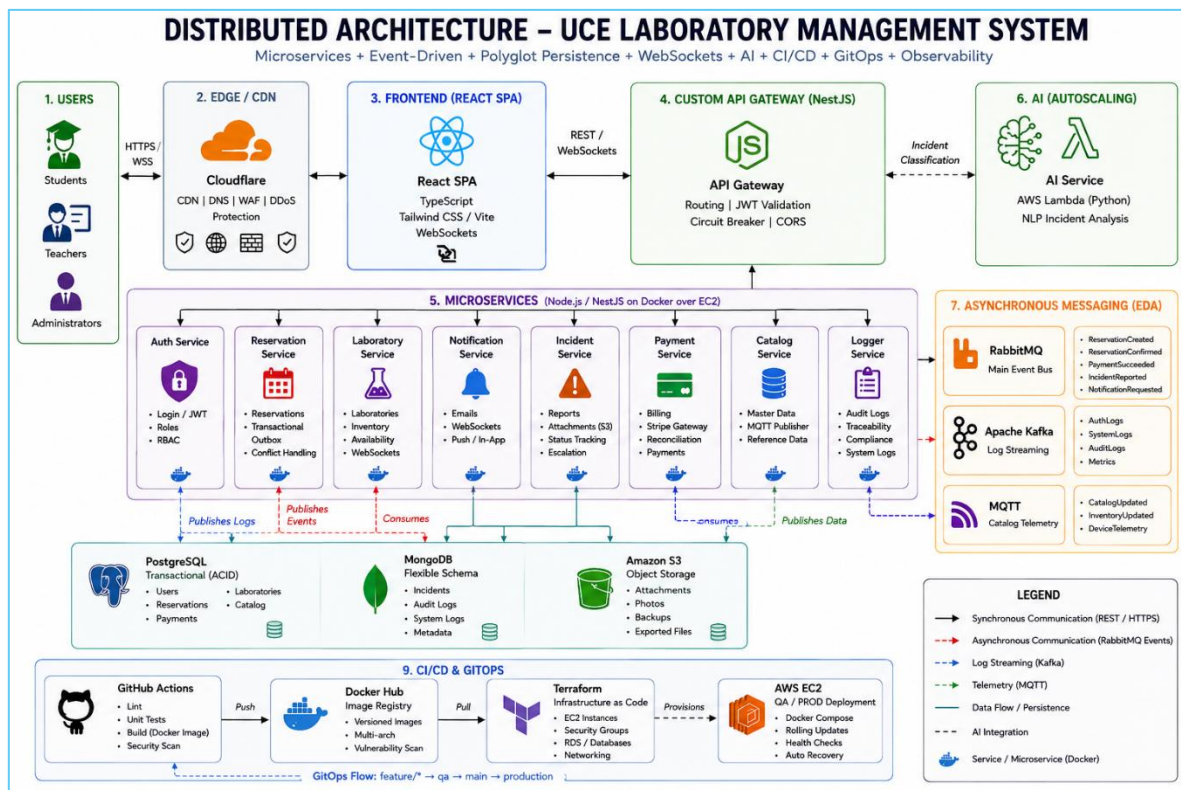


Diagram 1. General Architecture

4. Technologies Used

The technology stack is composed of industry-standard tools, optimized for enterprise environments:

Frontend: React, TypeScript, Vite, Tailwind CSS.

Backend: Node.js, NestJS.

Persistence: PostgreSQL (TypeORM), MongoDB (Mongoose), Amazon S3.

Messaging: RabbitMQ, Apache Kafka, MQTT.

Artificial Intelligence: Python (FastAPI/AWS Lambda).

Infrastructure & DevOps: Docker, GitHub Actions, Terraform, AWS (EC2, ALB, ECR, S3).

5. Monorepo Architecture

The source code is managed through a monorepo structure that unifies version control and facilitates Continuous Integration (CI/CD). The root distributes artifacts into semantic directories: apps/frontend concentrates the user interface; services independently hosts the API Gateway and microservices; and infra consolidates Terraform configurations, segmented into modules and environments.

6. Frontend

The client application is deployed as a robust SPA, structured to manage the complexity of academic roles.

Architecture: Global composition pattern through a unified session context (AuthContext).

Organization: The code is subdivided into clear layers: api for HTTP clients, context for global state persistence, layouts for visual segregation, and routes / pages for orchestration.

Flow: Access is restricted through protected routes (ProtectedRoute) that validate the presence of credentials and roles dynamically.

Communication with API Gateway: All outgoing REST traffic is channeled by a centralized Axios interceptor that attaches authorization (Bearer Token) and proactively resolves credential rotation. Real-time notification traffic (WebSockets) bypasses the gateway to connect directly and optimally to the notification service.

7. API Gateway

It functions as the epicenter of HTTP traffic, applying the unified gateway pattern without absorbing business logic.

- **Architecture:** Designed on NestJS as a high-performance reverse proxy.
- **Responsibilities:** Exposure of a single catalog of routes (/api/...), hiding the internal network topology.
- **Routing:** Introduces end-to-end traceability by injecting correlation identifiers (x-correlation-id). Utilizes native streaming for incidents (supporting multipart) and standard synchronous consumption for the rest of the resources.
- **Security:** Acts as a passive forwarder of the authorization header. It does not locally validate the JWT signature, delegating this responsibility directly to the corresponding domain.
- **Communication with Microservices:** Implements Circuit Breaker patterns selectively on critical routes to avoid cascading saturation due to internal network latencies.

8. Microservices

8.1. API Gateway: It acts as the single point of entry for client applications. It implements the API Gateway pattern to seamlessly route HTTP requests to the corresponding backend microservices, manage CORS policies, and abstract the internal network topology. This prevents the frontend from needing to know the individual IP addresses or ports of each service.

8.2. Auth Service: It is the identity authority of the system. Its architecture separates credential management from user administration. It uses bcrypt for password encryption and issues JWT tokens without maintaining state in memory (stateless). It persists its entities in PostgreSQL, guaranteeing strong referential integrity.

8.3. Reservation Service: Responsible for the business core. It implements a strict rules engine to avoid schedule collisions through pessimistic locks in TypeORM. It stands out for using the Transactional Outbox pattern, guaranteeing that the creation of the reservation and the intention to notify are executed in a single atomic transaction in PostgreSQL. A background process reads this table and propagates reliable events to RabbitMQ.

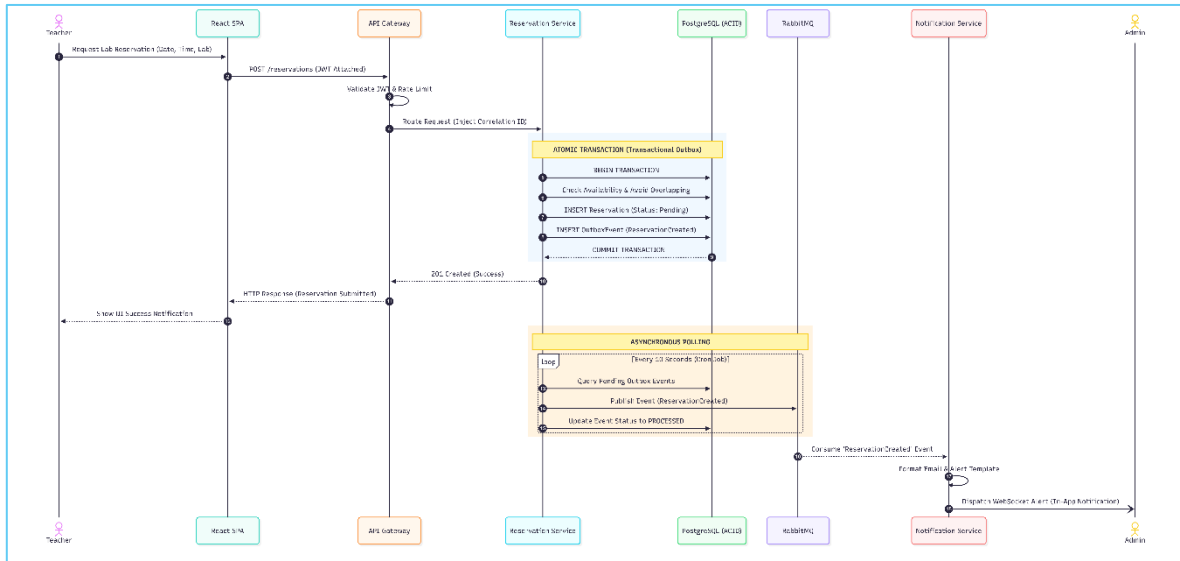


Diagram 2. Sequence Diagram - Transactional Outbox

8.4. Laboratory Service: Controls the physical operational state of academic resources (e.g., active, under maintenance). Its flow is tightly intertwined with external asynchronous events: through a RabbitMQ consumer, it reacts to reported incidents to proactively degrade the availability of a resource and avoid inventory collisions.

8.5. Notification Service: A hybrid microservice focused on multichannel distribution. It lacks complex mutation controllers; instead, it relies on asynchronous consumers listening to RabbitMQ and Kafka. Once an event is ingested (e.g., an approved reservation), it persists the trace in PostgreSQL and pushes the update to the client via WebSockets for instant feedback, or delegates to SMTP for formal emails.

8.6. Catalog Service: Exposes the quick view of laboratories, updating its inventory in a relational cache based on operational events. It includes an MQTT Publisher for IoT device telemetry and real-time monitoring.

8.7. Incident Service: Handles hybrid persistence. It saves incident metadata in MongoDB and delegates physical damage images and evidence directly to AWS S3. Upon registering a new incident, this service communicates synchronously with the AI Service to obtain a severity classification before dispatching alerts to the network.

8.8. Logger Service: Acts as a total observer by subscribing to the RabbitMQ wildcard (#) and Kafka topics. Every event propagated across the network is captured and saved in MongoDB, forming an immutable forensic audit log.

8.9. Payment Service: Serves as an Anti-Corruption Layer (ACL) to delegate and verify the billing of Premium laboratories through external payment gateways (Stripe), isolating the external domain logic from the system's core.

8.10. AI Service (AWS Lambda): A serverless function written in Python and invoked via HTTP API. It analyzes the incident report text using Natural Language Processing (NLP) to classify the severity and category, optimizing infrastructure resources by executing strictly on-demand.

9. Cloud Infrastructure

The productive infrastructure is coded entirely using Terraform, orchestrating resources in AWS under a paradigm of high modular cohesion.

- **Module Organization:** Dedicated directories for VPC, ALB, EC2, Bastion, and ECR.

- **VPC and Networks:** Strict isolation. Processing nodes in private subnets, exposed only through public load balancers.
- **Auto Scaling and Load Balancer:** The cluster relies on an Auto Scaling Group of EC2 instances, governed by an Application Load Balancer (ALB) that evenly distributes incoming requests across multiple Availability Zones (Multi-AZ).
- **IAM and Security Groups:** Use of Instance Profiles so that instances access attached services (like S3) without storing physical passwords. Firewall rules are airtight, allowing SSH only through the Bastion instance.
- **Environment Variables:** Resolved and injected cryptographically from GitHub Secrets at deployment time, ensuring source code immutability.

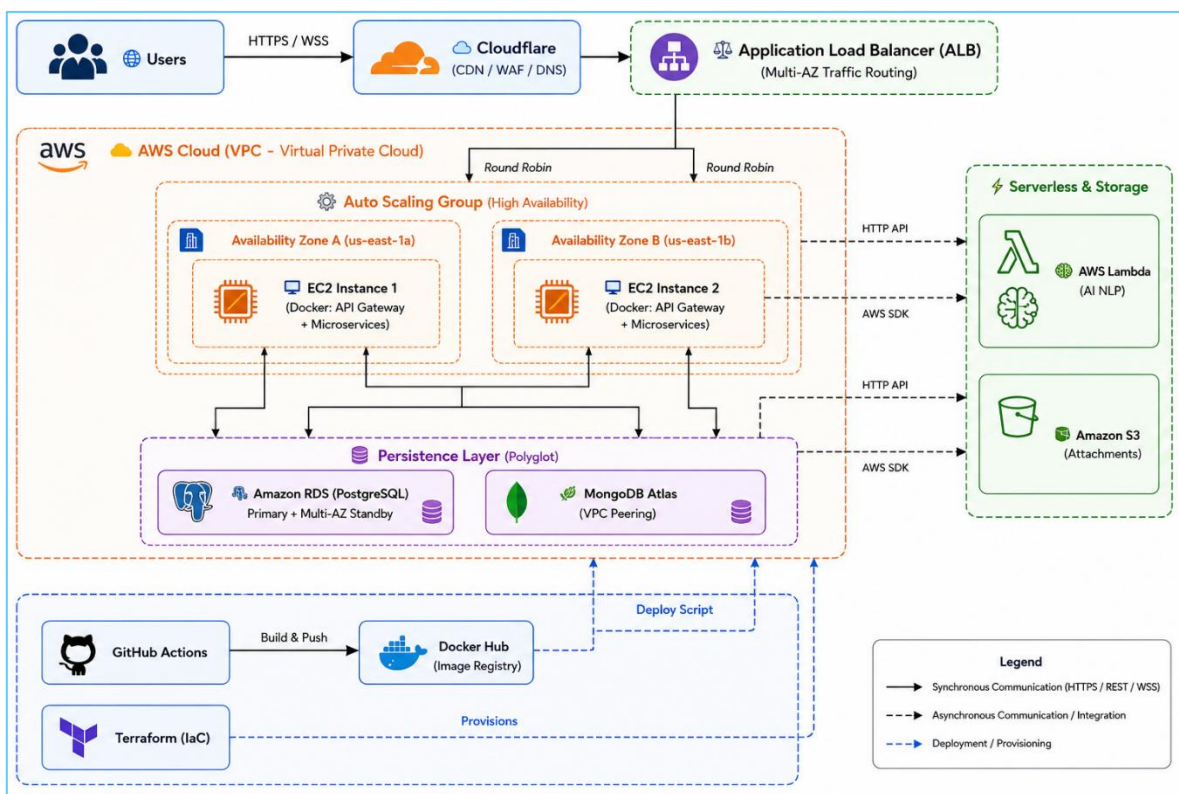


Diagram 3. Cloud HA Architecture

10. Containerization

Portability is resolved universally with Docker.

- **Dockerfiles and Compose:** Each component features an optimized multi-stage build for Node.js. Local and deployment orchestration is handled through multiple docker-compose manifests.
- **Networks and Communication:** Bridge-type network topology, allowing internal DNS resolution between containers by service name. Boot dependencies are managed with explicit health checks (e.g., waiting for Postgres to be ready before starting the application).

11. Continuous Integration and Deployment (CI/CD)

The ecosystem implements a robust enterprise-grade pipeline using GitHub Actions.

- **CI, Build, and Test:** DRY architecture with reusable workflows (_service-ci.yml). Selectively detects which microservice underwent changes. Executes linters, coverage tests, and static analysis via SonarQube.
- **CD and Publishing:** Builds and signs images, uploading them to Amazon ECR/Docker Hub.
- **QA and Production Deployment:** Dynamically discovers the active IPs of the Auto Scaling Group via AWS CLI, channeling deployments through the Bastion Host. The nodes execute a rolling update, downloading images and initializing them safely in EC2.

12. Security

The perimeter is shielded by strict stateless policies.

- **JWT and Refresh Token:** Identity travels signed in headers. The Refresh Token allows credential renewal in the frontend interceptor without forcing user disconnection.
- **Authorization and Roles (RBAC):** Hierarchies (Admin, Teacher, Student) validated natively in NestJS through RolesGuard, rejecting access in memory before hitting the database layer.
- **AWS IAM and Secrets:** Total absence of credentials in the code. Intensive use of IAM profiles for authorization between AWS resources and Secrets for the CI/CD pipeline.

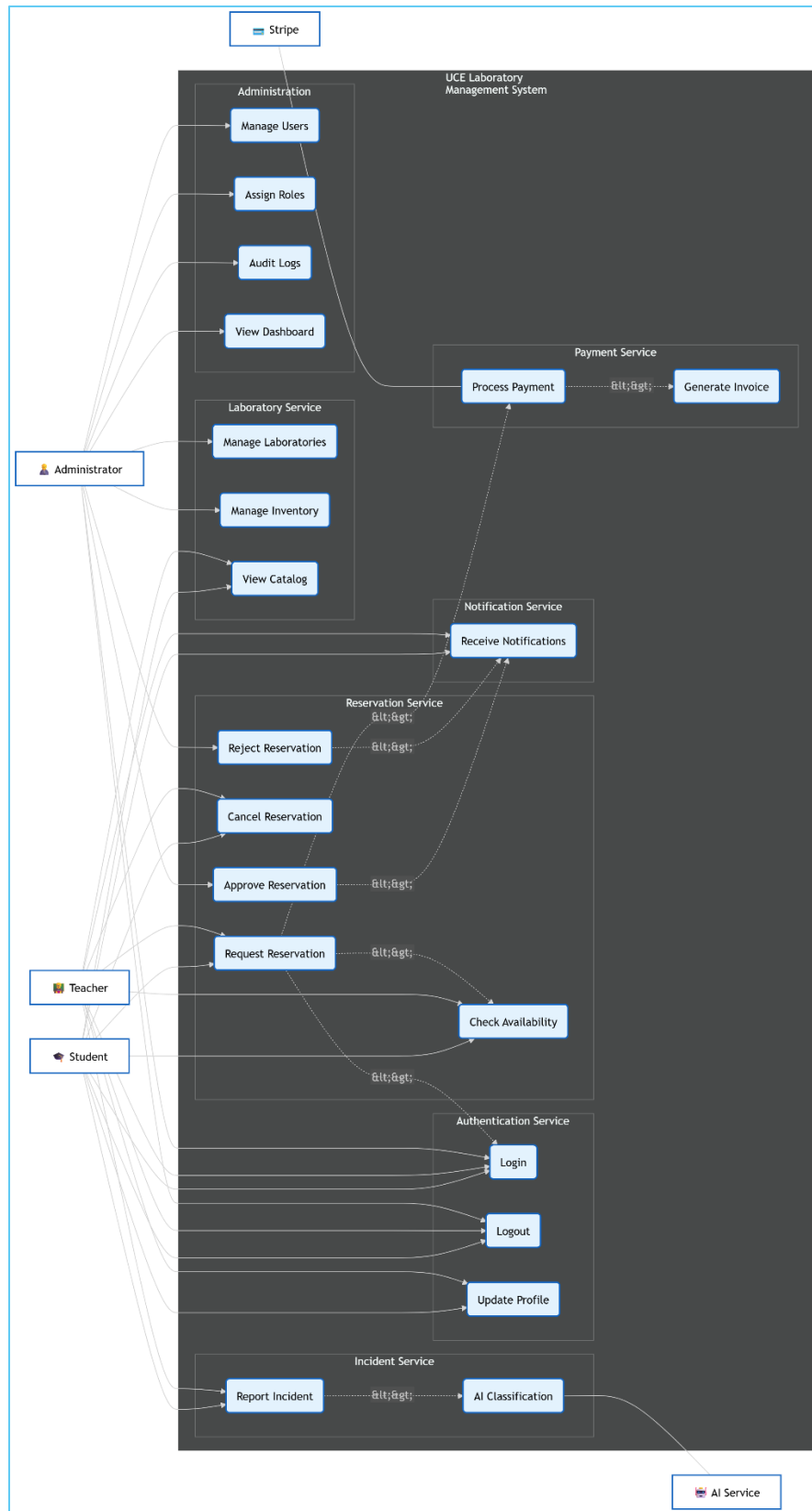


Diagram 4. Use Case Diagram - Detailed RBAC

13. Communication Between Components

The architecture balances synchronous responses at the perimeter and asynchronous load dissipation internally:

- **Hybrid Transactional Flow:** Frontend → (HTTPS) → API Gateway → (HTTP/REST Synchronous) → Reservation Service → (PostgreSQL Commit Synchronous + Outbox) → (Return to Client). Simultaneously: Outbox → RabbitMQ → Notification Service → (WebSocket) → Frontend.

14. Data Persistence

Polyglot and isolated pattern (Database-per-service).

- **PostgreSQL:** Sustained with TypeORM for Auth, Reservation, Laboratory, Catalog, and Payment. Allows strict ACID control.
- **Relationships:** Strong isolation. Cascading failures do not cross microservice boundaries, keeping foreign keys contained per domain.
- **MongoDB:** Flexibility for dynamic collections (Logs, Incidents).
- **Amazon S3:** Durable and scalable object storage for incident attachments and system backups.

15. Observability

The system is inherently opaque to the client but transparent to the operator.

- **Logs:** Correlated by x-correlation-id.
- **Audit Monitoring:** The Logger Service absorbs all historical load via messaging.
- **Health Checks:** Native NestJS control points (Terminus) exposed to the load balancer to orchestrate automated restarts or evictions of degraded instances.

16. Scalability and High Availability

The total lack of temporary session state in memory (Stateless HTTP) by the services empowers the Auto Scaling Group to destroy and instantiate EC2 servers according to demand (e.g., registration periods). Diverting costly tasks to RabbitMQ prevents the Node.js event loop from collapsing, maintaining high operational availability against I/O peaks.

17. Complete System Operation

End-to-End Walkthrough:

- 1. Login:** The user submits data. The Gateway forwards to Auth Service. PostgreSQL verifies the bcrypt hash. JWT is issued. The frontend stores and reconstructs the navigation tree based on the role.
- 2. Laboratory Query:** The browser requests the Gateway. It routes to the Catalog Service. The synchronous query is dispatched in milliseconds from PostgreSQL to the React SPA.
- 3. Reservation Creation:** The complex payload enters the Reservation Service. An optimistic lock is executed in the DB, the reservation is inserted, and the notification is inserted into the Outbox table. A 201 Created response is sent to the frontend.
- 4. Notification:** In the background (10ms later), the Outbox processor reads the table and throws the message to RabbitMQ. The Notification Service intercepts it, records its log, and via its WebSocket server permanently connected to the frontend, pushes the visual toast alerting that the reservation has been approved.

18. Architectural Decisions

- **Why Microservices?:** Guarantees independent scaling (e.g., scaling reservations during exam season without affecting authentication), favors fault isolation, and divides the database to prevent contention (massive locks).
- **Why NestJS and React?:** NestJS provides an opinionated architecture and strict typing (TypeScript), crucial for maintaining homogeneity across more than 8 services. React delivers the broadest and most reactive ecosystem for a fluid management SPA.
- **Why PostgreSQL?:** Critical requirement of ACID transactional compliance to avoid double-booking physical laboratories.
- **Why RabbitMQ instead of direct HTTP requests?:** To implement Event-Driven Architecture (EDA). It decouples the logic, ensuring that a temporary failure of the email service does not prevent a reservation from being successfully confirmed.
- **Why Serverless AI (AWS Lambda)?:** Prevents memory exhaustion on EC2 instances by isolating computationally heavy NLP tasks.
- **Why Terraform and AWS?:** Terraform establishes immutable infrastructure, eliminating drift between QA and Production. AWS guarantees the corporate SLA support necessary for the physical instances, networks, and load balancers required by the containers.

19. Risks and Limitations

- **Central Dependency on RabbitMQ:** As the asynchronous backbone, its saturation or crash would halt notifications and inter-domain synchronizations, although the base system would continue operating.

- **AWS Academy Credential Rotation:** The limitations of academic accounts cause short-term obsolescence of deployment tokens, requiring frequent operational maintenance in CI/CD repositories.
- **Distributed Network Latency:** Multiple hops between containers (Gateway → Internal Service) add a slight increase in basal latency compared to a traditional monolith.
- **Cost and Operational Complexity:** Maintaining an EC2 cluster, Bastion, separate databases, and an ALB carries a high financial overhead and requires mature DevOps knowledge to diagnose transactional flows.

20. Project Metrics

- **Languages:** TypeScript, JavaScript, Python, SQL, HCL (Terraform), Shell.
- **Frameworks:** NestJS, React, Vite, Tailwind CSS.
- **Backend Microservices: 9** (Gateway, Auth, Reservation, Laboratory, Catalog, Notification, Payment, Incident, Logger).
- **Serverless Functions: 1** (AI Service).
- **Docker Images Created: 10** (9 Backend + 1 Frontend).
- **Workflows (CI/CD):** 17 independent GitHub Actions pipelines.
- **Core Infrastructure (AWS):** 2+ EC2 Instances, 1 Application Load Balancer, 1 Bastion Host, 1 Auto Scaling Group, 1 S3 Bucket.
- **Storage (Databases):** 7 Isolated clusters/schemas (5 PostgreSQL, 2 MongoDB).
- **Quality Analysis:** Static coverage and continuous inspection via SonarQube.
- **Source Code Volume (estimated):** +25,000 lines distributed in the monorepo.

21. AWS Infrastructure Budget Estimate

This report details the estimated costs for deploying the architecture in a commercial AWS account, operating 24/7 (730 hours per month) in the us-east-1 (N. Virginia) region.

1. QA Environment (Current)

This reflects your exact environments/qa/main.tf configuration (desired_capacity = 1).

Resource	Technical Description	Unit Cost	Monthly Cost
EC2 Instance (App)	1x <code>t3.medium</code> (2 vCPU, 4GB RAM) for microservices	\$0.0416 / hour	\$30.36
EC2 Instance (Bastion)	1x <code>t3.micro</code> (2 vCPU, 1GB RAM) for SSH access	\$0.0104 / hour	\$7.59
EBS Volumes	~30GB (App) + 8GB (Bastion) gp3 SSDs	\$0.08 / GB-month	\$3.04
App Load Balancer	1 Base ALB + ~1 LCU (moderate traffic)	\$0.0225 / hour	~\$22.26
NAT Gateway	1 NAT Gateway (Single-AZ)	\$0.0450 / hour	\$32.85
NAT Data Processing	Data traffic (~10GB/month)	\$0.045 / GB	~\$0.45
Amazon ECR	Docker image storage (~4.5 GB)	\$0.10 / GB-month	\$0.45
Amazon S3	Terraform State & Incidents Bucket (~5 GB)	\$0.023 / GB-month	\$0.12
AWS API Gateway	AI Service API (~10,000 requests/month)	\$3.50 / 1M reqs	~\$0.04
AWS Lambda	AI Service Compute (2s duration, 512MB RAM)	Free Tier	\$0.00

Total Monthly (QA): ~\$97.16 USD Total Annual (QA): ~\$1,165.92 USD

2. Production Environment Projection (Multi-AZ)

For a real production environment, the Terraform code has variables prepared for High Availability (enable_rds = true, min_size = 2). This would be the enterprise cost:

Resource	Technical Description	Monthly Cost
EC2 (High Availability)	2x <code>t3.medium</code> distributed across 2 Availability Zones	\$60.72
EC2 Instance (Bastion)	1x <code>t3.micro</code> for management	\$7.59
EBS Volumes	~60GB (App) + 8GB (Bastion) gp3 SSDs	\$5.44
App Load Balancer	1 Base ALB + ~2 LCU (production traffic)	~\$28.00
NAT Gateway	1x NAT Gateway (Based on your <code>vpc</code> module)	\$33.85
Amazon S3 & ECR	Images and Buckets (~15 GB total)	\$0.80
AWS API Gateway	AI Service API (~1,000,000 requests/month)	\$3.50
AWS Lambda	AI Service Compute (~1,000,000 requests, 2s avg)	~\$2.00
Amazon RDS (Database)	Currently disabled in code (<code>enable_rds = false</code>)	\$0.00

Total Monthly (Production): ~\$192.01 **USD** **Total Annual (Production):** ~\$2,304.12 **USD**.

Budgeting Recommendations & Terraform Insights

1. **NAT Gateway Bottleneck:** Your `vpc` module currently provisions only 1 NAT Gateway for both private subnets. While this saves \$32.85/month in Production, it is a single point of failure. If that Availability Zone goes down, your EC2 instances in the other zone will lose internet access.
2. **Databases in Production:** Your Production variables default to `enable_rds = false` (running databases inside Docker). If you plan to switch to Amazon RDS (`db.t3.micro`) for managed databases in production, add ~\$15.00/month to the Production budget.
3. **AWS Lambda Perpetual Free Tier:** AWS Lambda includes a *perpetual* free tier of 1 million requests and 400,000 GB-seconds of compute time per month. This means your AI Service compute will likely remain completely free or cost pennies even in a commercial production account, as long as it stays under those limits.

- 4. AWS Free Tier:** If this is a completely new commercial account, the first year will include 750 hours of EC2 t2/t3.micro and the Load Balancer covered by the Free Tier, reducing the cost drastically for the first year.

22. Implemented Best Practices

The project strictly adheres to the design pillars of The Twelve-Factor App, highlighting the injection of volatile configurations by external environments, absolute parity between development and production through identical containers, and the design of stateless processes to favor self-concurrency and cloud scalability.

23. Possible Future Improvements

Three opportunities for architectural maturation are identified:

Migrate the validation and perimeter expiration of the JWT token to the API Gateway itself, preventing traffic with revoked credentials from penetrating the deep internal network.

Transition from on-the-fly schema synchronization (`synchronize: true`) in TypeORM to controlled DDL/DML migrations, indispensable for mission-critical environments.

Adopt a managed container solution (Amazon ECS Fargate or EKS) replacing the artisanal orchestration of Docker Compose on raw EC2 instances, minimizing the effort of patching operating systems.

24. Conclusions

The design and implementation of the UCE Lab Management System expose a degree of maturity comparable to contemporary enterprise platforms. The segmentation of the monorepo into isolated functional units, interconnected through asynchronous messaging (RabbitMQ) and coordinated by automated CI/CD flows (GitHub Actions + ECR), justifies the technical investment and modeling effort.

Far from over-engineering, the decision to use microservices elegantly solves the independent scalability of high-friction domains (such as temporary reservations), protecting the overall performance of the system. This software product successfully consolidates theoretical advanced software engineering knowledge with the pragmatic realities of Cloud Computing and DevOps culture.